

What is Flexbox?

Flexbox (Flexible Box Layout) is a one-dimensional CSS layout model that provides an efficient way to arrange, align, and distribute space among items in a container. It excels at building rows OR columns of content that need to wrap, align, and distribute space dynamically.

■ *Flexbox is one-dimensional — it handles layout in a single axis at a time (row or column). For two-dimensional layouts, consider CSS Grid.*

Quick Start

Enable flexbox on any container with a single declaration:

```
.container {  
  
  display: flex;  
  
}
```

All direct children immediately become **flex items**. No other setup required.

CONTAINER PROPERTIES

flex-direction

Defines the main axis — the direction flex items are placed.

row	Items placed left → right (default)	row row-reverse
column	Items stacked top → bottom	column column-reverse

```
.container {  
  
  display: flex;  
  
  flex-direction: row;      /* default */  
  
  /* flex-direction: column; */  
  
}
```

flex-wrap

Controls whether items wrap onto multiple lines when they overflow.

nowrap	All items on one line (default)	nowrap
wrap	Items wrap to next line	wrap
wrap-reverse	Items wrap upward	wrap-reverse

```
.container {  
  
  flex-wrap: wrap;  
  
}
```

flex-flow

Shorthand for **flex-direction** + **flex-wrap** combined.

```
.container {  
  
  flex-flow: row wrap; /* direction | wrap */  
  
}
```

justify-content

Aligns items along the **main axis** (horizontal by default).

flex-start	Pack items toward start (default)	flex-start
flex-end	Pack items toward end	flex-end
center	Center items	center
space-between	Equal gaps between items	space-between
space-around	Equal space around items	space-around
space-evenly	Equal space between & edges	space-evenly

```
.container {  
  
  justify-content: space-between;  
  
}
```

align-items

Aligns items along the **cross axis** (vertical by default).

stretch	Items stretch to fill container (default)	stretch
flex-start	Align to start of cross axis	flex-start
flex-end	Align to end of cross axis	flex-end
center	Center on cross axis	center
baseline	Align text baselines	baseline

```
.container {

    align-items: center;

}
```

align-content

Aligns **rows** of items along the cross axis (only applies when wrapping).

flex-start	Rows packed to start	flex-start
flex-end	Rows packed to end	flex-end
center	Rows centered	center
space-between	Equal gap between rows	space-between
space-around	Equal space around rows	space-around
stretch	Rows stretch to fill (default)	stretch

```
.container {

    flex-wrap: wrap;

    align-content: space-between;

}
```

gap / row-gap / column-gap

Sets spacing between flex items (no space on outer edges).

```
.container {

    gap: 16px;           /* row and column gap */

}
```

```
gap: 12px 24px;      /* row-gap | column-gap */

row-gap: 12px;

column-gap: 24px;

}
```

ITEM PROPERTIES

order

Controls the order items appear. Default is 0. Lower values appear first.

```
.item-first { order: -1; }

.item-last  { order: 1; }
```

flex-grow

Defines how much a flex item should grow relative to siblings when extra space is available.

```
.item-a { flex-grow: 1; } /* takes 1 share */

.item-b { flex-grow: 2; } /* takes 2 shares (2x wider) */

.item-c { flex-grow: 0; } /* does not grow (default) */
```

flex-shrink

Defines how much a flex item shrinks relative to siblings when space is tight.

```
.item { flex-shrink: 1; } /* shrinks normally (default) */

.item { flex-shrink: 0; } /* never shrinks */

.item { flex-shrink: 2; } /* shrinks 2x faster */
```

flex-basis

Sets the initial main-size of an item before growing/shrinking is applied.

```
.item { flex-basis: auto; } /* uses content size (default) */

.item { flex-basis: 200px; } /* fixed starting width */

.item { flex-basis: 30%; } /* relative starting width */
```

```
.item { flex-basis: 0;      } /* ignore content size */
```

flex (shorthand)

Shorthand for **flex-grow**, **flex-shrink**, and **flex-basis**.

```
.item { flex: 1;           } /* grow:1 shrink:1 basis:0 */  
  
.item { flex: 0 1 auto;    } /* default */  
  
.item { flex: 2 1 200px;   } /* grow:2 shrink:1 basis:200px */  
  
.item { flex: none;        } /* 0 0 auto – rigid item */
```

■ *flex: 1 is the most common shorthand. It makes items share available space equally.*

align-self

Overrides **align-items** for a single item.

auto	Inherit parent align-items (default)	auto
flex-start	Align to start	flex-start
flex-end	Align to end	flex-end
center	Center on cross axis	center
stretch	Stretch to fill	stretch
baseline	Align to text baseline	baseline

```
.special-item {  
  
    align-self: flex-end;  
  
}
```

COMMON PATTERNS

Perfect Centering

Center any element both horizontally and vertically:

```
.container {  
  
    display: flex;
```

```
    justify-content: center;

    align-items: center;

    height: 100vh;

}
```

Navigation Bar

Logo on the left, links on the right:

```
.navbar {

    display: flex;

    justify-content: space-between;

    align-items: center;

    padding: 16px 24px;

}

.nav-links {

    display: flex;

    gap: 24px;

}
```

Card Grid with Wrap

Equal-width cards that wrap responsively:

```
.card-grid {

    display: flex;

    flex-wrap: wrap;

    gap: 16px;

}
```

```
.card {  
  
    flex: 1 1 280px;    /* grow, shrink, min 280px */  
  
}
```

Sticky Footer Layout

Footer always sticks to the bottom of the viewport:

```
body {  
  
    display: flex;  
  
    flex-direction: column;  
  
    min-height: 100vh;  
  
}  
  
main { flex: 1; }    /* main grows, footer stays */
```

Holy Grail Layout

Classic header / sidebar / content / sidebar / footer:

```
.page {  
  
    display: flex;  
  
    flex-direction: column;  
  
    min-height: 100vh;  
  
}  
  
.body {  
  
    display: flex;  
  
    flex: 1;  
  
}
```

```
aside      { width: 220px; }

main      { flex: 1; }
```

QUICK CHEAT SHEET

Property	Applies To	Key Values
display	Container	flex inline-flex
flex-direction	Container	row column row-reverse column-reverse
flex-wrap	Container	nowrap wrap wrap-reverse
flex-flow	Container	<direction> <wrap>
justify-content	Container	flex-start center space-between space-around space-between
align-items	Container	stretch flex-start flex-end center baseline
align-content	Container	stretch flex-start flex-end center space-between
gap	Container	<length> <length> <length>
order	Item	<integer> (default: 0)
flex-grow	Item	<number> (default: 0)
flex-shrink	Item	<number> (default: 1)
flex-basis	Item	auto <length> <percentage>
flex	Item	<grow> <shrink> <basis> 1 none
align-self	Item	auto flex-start flex-end center stretch

TIPS & BEST PRACTICES

Browser Support
Flexbox is supported in all modern browsers (Chrome, Firefox, Safari, Edge). Prefix-free syntax works everywhere. No polyfills needed for production.

Flexbox vs Grid
Use **Flexbox** for one-dimensional layouts: nav bars, button groups, centering, card rows. Use **CSS Grid** for two-dimensional layouts: full page structure, complex magazine-style grids. They work great together.

Debugging Tips
Open browser DevTools and hover over a flex container — you'll see a 'flex' badge. Click it to toggle a flex overlay that visualizes axes, gaps, and item sizes. Firefox DevTools has particularly rich flex debugging.

■ *min-width: 0 on flex items fixes overflow issues when items contain long text or images.*